UNEB	DCET-I Sistemas de Informação Prof$_{a.}$: Maria Inés Restovic	Linguagem de Programação I Trabalho Final	 Data:20/10/2026

RoboLab: Desafio de Sobrevivência

1. Apresentação

Uma instalação tecnológica experimental foi invadida por robôs hostis. Um robô explorador, controlado pelo jogador, deverá percorrer diferentes setores, superar obstáculos, evitar inimigos e alcançar a saída. O cenário será representado por uma matriz bidimensional. Deverão ser implementados elementos obrigatórios e três obstáculos originais, com diferentes níveis de complexidade.

2. Objetivos de aprendizagem

- Representar cenários com matrizes e utilizar arrays de estruturas.
- Modelar robôs, inimigos e obstáculos por meio de struct.
- Aplicar funções, passagem de parâmetros por valor e referência, condicionais e laços.
- Implementar movimentação, detecção de colisões e regras de interação.
- Ler e validar arquivos texto e manipular registros em arquivo binário.
- Integrar algoritmos de C++ com desenho e captura de teclado por meio da raylib.

3. Cenário matricial

Cada fase terá matriz de pelo menos 30 linhas por 30 colunas. O valor de cada célula identifica um elemento fixo do ambiente. Personagens móveis devem ter suas posições mantidas separadamente da matriz.

Código	Significado
0	Caminho livre
1	Parede fixa
2	Campo elétrico
3	Estação de energia
4	Saída
5	Obstáculo especial
6	Área de perigo

A equipe deverá definir a representação de obstáculos adicionais sem criar ambiguidades entre os códigos. É permitido usar formas geométricas e cores em vez de imagens externas.

4. Robô do jogador e movimentação

```
struct Robo {
    int linha;
    int coluna;
    int energia;
    int vidas;
    int pontos;
};
```

O robô inicia com três vidas e 100 unidades de energia. O teclado desloca o robô uma célula por comando. Não é permitido atravessar paredes ou sair da matriz. Energia igual a zero provoca perda de uma vida; caso ainda existam vidas, o robô reinicia a fase com energia restaurada. O jogo termina ao perder todas as vidas ou ao concluir a última fase.

5. Obstáculos obrigatórios

Obstáculo	Nível	Comportamento obrigatório
Parede fixa	Básico	Bloqueia a passagem.
Campo elétrico	Básico	Ao entrar, retira energia conforme regras.txt.
Estação de energia	Básico	Recupera energia até o máximo permitido; uso único por estação.
Robô patrulheiro	Intermediário	Move-se entre limites definidos e inverte a direção ao encontrar bloqueio.
Robô perseguidor	Avançado	Quando o jogador está no alcance configurado, tenta aproximar-se sem atravessar paredes; não se exige busca de caminho ótimo.

Cada contato com robos inimigos provoca danos, a equipe pode definir o tipo de perda de energia ouvida nas regras.

6. Obstáculos originais

Cada estudante desenvolverá três obstáculos próprios, obrigatoriamente com comportamentos distintos dos já exigidos: um básico, um intermediário e um avançado. Alterar apenas aparência, cor ou valor de dano não caracteriza obstáculo original.

Nível	Exemplo	Critério de complexidade
Básico	Piso escorregadio	Regra simples dependente da célula.
Intermediário	Porta temporizada	Comportamento que muda com o tempo ou estado.
Avançado	Sentinela que dispara	Combinação de detecção, deslocamento e interação.

A equipe deverá descrever previamente os três obstáculos propostos, indicando sua lógica e justificando a dificuldade. A professora poderá solicitar ajustes para equilibrar a complexidade entre os trabalhos.

7. Fases e regras de progressão

Elemento	Fase 1	Fase 2	Fase 3
Paredes, campos e energia	Sim	Sim	Sim
Patrulheiros	Não	Sim	Sim
Perseguidores	Não	Não	Sim
Obstáculo original	Básico	Intermediário	Avançado

Cada fase deverá apresentar um cenário diferente. A fase é concluída ao alcançar a saída; a terceira fase encerra o jogo com vitória. A pontuação será calculada segundo as regras de configuração.

8. Arquivo texto de regras — regras.txt

O programa apresentara ao usuário o roteiro e regras do jogo e de cada fase em um ou mais arquivos txt.

9. Arquivos texto dos cenários

As três matrizes deverão ser carregadas dos arquivos fase1.txt, fase2.txt e fase3.txt. Cada arquivo conterá exatamente 30 linhas de 30 dígitos (ou dimensões maiores, se documentadas), sem separadores, utilizando os códigos definidos. O programa deverá verificar dimensões, caracteres permitidos, posição inicial válida e existência de saída. É recomendável verificar também se a saída é alcançável.

Trecho ilustrativo de uma matriz menor (não substitui as dimensões mínimas exigidas):

```
1111111111
1000000001
1011100101
1000100001
1100101101
1000000101
1000100001
1111111111
```

10. Arquivo binário de estatísticas — estatisticas.dat

O jogo deverá manter um registro por jogador, atualizado após cada partida. O nome será utilizado como chave de busca; o programa deverá impedir cadastros duplicados com o mesmo nome, de acordo com uma regra de comparação documentada.

```
struct Estatistica {
    char nome[40];
    int partidasJogadas;
    int vitorias;
    int derrotas;
    int maiorPontuacao;
    int fasesConcluidas;
};
```

Operações obrigatórias: cadastrar jogador; buscar pelo nome; atualizar estatísticas ao encerrar partida; listar registros; exibir ranking ordenado pela maior pontuação. O campo fasesConcluidas representa o total acumulado de fases concluídas em todas as partidas. Partidas interrompidas devem ter tratamento documentado.

11. Interface gráfica com raylib (OPCIONAL)

- Exibir matriz, paredes, perigos, estações, saída, robôs e obstáculos originais.
- Capturar comandos de teclado para deslocamento em células.
- Mostrar vidas, energia, pontos e fase atual.
- Apresentar mensagens de vitória, derrota e erros de carregamento.
- Manter a janela responsiva; não utilizar esperas bloqueantes no laço principal.

São suficientes primitivas como DrawRectangle(), DrawCircle(), DrawText(), IsKeyPressed() e o ciclo BeginDrawing()/EndDrawing(). Sprites e sons são opcionais.

12. Organização e modularização

A lógica de movimentação, colisão, arquivos e regras deve ser organizada em funções próprias, evitando concentrar toda a implementação em main().
Sugestões de funções:

```
void desenharCenario();
void desenharRobo(const Robo &robo);
void moverRobo(Robo &robo, int direcao);
void atualizarInimigos();
void verificarColisoes(Robo &robo);
void aplicarObstaculos(Robo &robo);
bool verificarFimDeJogo(const Robo &robo);
```

As assinaturas são ilustrativas e deverão receber os parâmetros necessários no projeto real. É recomendado separar o programa em main.cpp, jogo.cpp/.h, robo.cpp/.h, obstaculos.cpp/.h e arquivos.cpp/.h.

13. Fluxo esperado de execução

1. Ler regras.txt
2. Identificar ou cadastrar jogador em estatisticas.dat.
3. Carregar a matriz da fase.
4. Executar movimentação, obstáculos, inimigos e pontuação na janela raylib.
5. Registrar o resultado da partida no arquivo binário.
6. Exibir estatísticas e classificação.

14. Entrega e apresentação

- Código-fonte completo;
- Arquivos regras.txt;
- Arquivo estatisticas.dat;

- README com dependências, comandos de compilação, controles, regras e descrição dos obstáculos originais.
- Demonstração individual do funcionamento e explicação dos principais algoritmos.

15. Critérios de avaliação

Critério	Pontos
Cenário matricial e movimentação	1,5
Obstáculos obrigatórios	1,5
Três obstáculos originais	2,0
Matrizes, structs, arrays e funções	1,5
Leitura e validação dos arquivos texto	1,0
Arquivo binário: cadastro, consulta e atualização	1,0
Interface gráfica (opcional) e integração	1,0
Organização e legibilidade	0,5
TOTAL	10,0

Distribuição sugerida dos obstáculos originais: básico 0,5; intermediário 0,5; avançado 1,0 ponto. A originalidade deverá ser avaliada pela lógica implementada, não pelo acabamento visual.

Data de apresentação 15/12/2025